Stoquify Whole-System Audit Execution Prompt — 2026-08-03

Evidence review package · 3 August 2026

Act as a multidisciplinary principal review team: system architect, cyber-security architect, senior frontend engineer, UI/UX specialist, product strategist, business logic analyst, and SaaS growth advisor.

Project and mission

Audit the current filesystem state of `E:\ohada saas\Focused projects\stoquify` as an enterprise Stoquify/AqStoqFlow SaaS platform. Review functionality, architecture, workflows, services, server actions, API routes, TanStack Query, UI/UX, Prisma data ownership, security, financial controls, integrations, testing, observability, performance, release readiness, and product completeness.

Give a candid, evidence-led assessment of what is strong, useful, weak, duplicative, dangerous, missing, or unsupported. Recommend the smallest credible path to a complete, modern, trustworthy product. This is an audit and planning assignment, not an implementation or certification assignment.

Operating rules

- Read `AGENTS.md` first.
- Preserve the dirty worktree. Do not reset, clean, revert, migrate, seed, deploy, rotate secrets, call providers, or mutate production-like state.
- Treat current executable source, schema, migrations, and safe runtime evidence as authoritative over plans, reports, graphs, and comments.
- Treat `graphify-out/` and prior audits as navigation and hypotheses. Revalidate every material claim against current source.
- Record branch, commit, timestamp, dirty-state counts, inspected surfaces, omissions, and environment limits.
- Cite file and line evidence for material conclusions.
- Keep confirmed findings, supported inferences, hypotheses, unverified claims, and blocked checks separate.
- Do not infer that a route, schema, test, or static gate proves user value, runtime correctness, production controls, or statutory compliance.
- Preserve service ownership, evidence lineage, idempotency, reconciliation, reversals, maker-checker controls, redaction, and fail-closed statutory behavior where proven.

Specialist tracks

Use bounded read-only specialists, with the lead agent owning final synthesis and contradiction resolution:

1. Architecture, service ownership, actions, APIs, Prisma, ADRs, dependencies, workers, and system flows.
2. Frontend, TanStack Query keys, cache context, mutations, invalidation, hydration, client/server boundaries, and performance.
3. Product, role workflows, UI/UX, accessibility, localization, onboarding, reporting, packages, and adoption evidence.
4. Security, tenant isolation, authentication, RBAC, entitlements, fresh authentication, maker-checker, audit, privacy, uploads, and abuse controls.
5. Accounting, POS, inventory, purchasing/AP, payments, reconciliation, payroll, close, evidence, country packs, and OHADA controls.

6. Reliability, integrations, provider truth, retries, leases, errors, observability, migration safety, CI/CD, rollback, and release gates.

Specialists must not edit source or overwrite reports.

Evidence precedence

1. Current source, schema, constraints, and migrations.
2. Safely reproduced behavior and focused passing tests.
3. Enforced repository gates and accepted ADRs.
4. Current documentation.
5. Historical audits and readiness artifacts.
6. Knowledge-graph relationships.
7. Comments, filenames, proposals, and inferred intent.

Minimum evidence

Inspect `app/`, `app/api/`, `actions/`, `services/`, `components/`, `hooks/`, `lib/`, `config/`, `prisma/`, `scripts/`, tests, CI workflows, localization, public product claims, `graphify-out/`, `what-next/`, `innovation/`, and the July 28 system-audit package.

Revalidate every prior P0/P1 as one of: still confirmed, partially remediated, fully remediated, regressed, superseded, no longer applicable, unverified, or blocked.

Required analysis

Architecture and workflows

- Inventory public, authenticated, dashboard, API, scheduler, worker, integration, export, and public-token boundaries.
- Map bounded contexts, service owners, Prisma models, transaction owners, emitted events, read models, external adapters, and cross-domain dependencies.
- Reconcile accepted ADRs with current implementation.
- Trace representative flows as:

```
route/page → component → query/action/API → guard → service/transaction → Prisma → event/outbox/ledger → worker/provider → read model → invalidation → UI
```

Trace POS, inventory, purchasing/AP, payment/reconciliation, accounting/close, HR/payroll, country packs, identity, public receipts/uploads, communication, and agent/copilot flows.

Services, actions, APIs, and data integrity

- Determine whether actions/routes orchestrate while services own rules and persistence.
- Verify tenant, permission, module, location/resource, fresh-auth, rate-limit, maker-checker, idempotency, transaction, audit, error, and reversal controls.
- Review tenant keys, compound relationships, money precision, sequences, immutable evidence, soft deletion, optimistic concurrency, outbox/inbox, indexes, migrations, backfills, and rollback.

TanStack Query

Create a query/mutation registry with key, consumer, backing action/service, tenant/context dimensions, filters, stale/gc policy, pagination, optimistic behavior, rollback, invalidation, server revalidation, UI states, and tests. Identify collisions, missing organization dimensions, stale reuse after context changes, incomplete invalidation, waterfalls, oversized responses, and client/server-cache conflicts.

Product and UI/UX

Map owner, cashier, branch manager, inventory, purchasing/AP, accountant, payroll, employee, compliance, and administrator journeys. Evaluate navigation, canonical/alias/legacy/demo routes, onboarding, empty/loading/error/denied/offline states, reports, freshness/provenance, EN/FR parity, responsive behavior, WCAG 2.2 AA, design-system adoption, perceived performance, and public-promise accuracy.

Security and operations

Review tenant isolation, authentication/session lifecycle, MFA, CSRF/origins/proxies, RBAC, entitlements, audit durability, privacy/redaction, upload safety, webhook signatures/replay, financial fraud paths, errors, logs/metrics/traces/alerts, worker recovery, provider failure, backup/restore, secrets, supply-chain controls, migration safety, and staged rollback.

SaaS completeness

Evaluate tenant setup, packages, subscriptions, entitlement lifecycle, provisioning/deactivation, collaboration, imports/exports, notifications, demo/trial truth, activation, time-to-value, telemetry, adoption, retention, support operations, release evidence, and upgrade paths. Mark absent commercial or behavioral evidence `MISSING` or `UNVERIFIED`.

Classification

Value: `STRONG`, `USEFUL`, `WEAK`, `USELESS`, `MISSING`; add `DANGEROUS` where trust, tenant, privacy, financial, statutory, or irreversible-action risk applies.

Maturity: 0 absent; 1 scaffolded/ad hoc; 2 partial; 3 consistently implemented; 4 controlled/test-backed; 5 runtime/release evidenced.

Severity: P0 critical trust/corruption boundary; P1 important workflow/control failure; P2 bounded correctness/maintainability/performance/accessibility issue; P3 hygiene/documentation/polish.

Confidence: confirmed, high, medium, low/hypothesis, unverified, blocked.

Every finding must contain ID, category, module/workflow, roles, evidence, reproduction/test, current and expected behavior, impact, likelihood, blast radius, severity, confidence, existing controls, recommendation, acceptance test, dependencies, effort, owner, and status.

Safe verification

Inspect scripts before execution. Begin with read-only validation and gates where output paths are unset:

```
git status --short
git diff --stat
npm run prisma:validate
npm run typecheck
npm run lint
npm run service:boundary:fail
npm run hard-delete:fail
npm run error:boundary:fail
npm run workflow:assurance:release-gate
```

Run focused mocked/unit tests for representative tenant, entitlement, maker-checker, idempotency, offline replay, cache invalidation, provider failure, worker lease, backfill, redaction/export, error recovery, payment truth, AP, and close behavior.

Do not run `policy:gates`, `verify:repo`, `verify:ci`, `verify:release`, migrations, resets, seeds, database-connected tests, unidentified runtime checks, authenticated destructive browser flows, provider calls, load/credential attacks, or report-writing gates without proven disposable infrastructure and explicit artifact authority. Mark unsafe checks `SKIPPED`, `BLOCKED`, or `UNVERIFIED` and explain the prerequisite.

Recommendation rules

- Prefer fixing, exposing, consolidating, relabeling, redirecting, or retiring existing capability before adding a page, dashboard, service, agent, or workflow layer.
- A new feature must name the role, recurring job/control, observed pain/evidence gap, measurable outcome, canonical owner, permission/entitlement boundary, dependencies, evidence threshold, and acceptance test.
- Put unvalidated ideas into discovery.
- Do not recommend AI where a deterministic rule, checklist, filter, notification, or controlled workflow is enough.
- Do not propose abstractions without demonstrated coupling, ownership, testing, or change-cost evidence.
- Keep recommendations incremental, reversible, tenant-safe, and compatible with append-only financial/audit evidence.

Required outputs

Save date-distinct artifacts under `docs/system-audit/` in Markdown and PDF, plus JSON where machine-readable registries are useful:

- Execution prompt
- Whole-system audit
- Functionality and surface registry
- Workflow atlas
- Findings register
- Completion roadmap
- Verification ledger

Include system map, service ownership, dependency map, route/action/job guards, Prisma ownership, TanStack matrix, role/permission/entitlement matrix, KPI semantics, UX/accessibility, integration reliability, ADR reconciliation, dependency risks, prior-audit delta, coverage/omissions, residual risk, and 3–7 narrow follow-up prompts.

Completion standard

The audit is complete only when coverage denominators and omissions are explicit; representative workflows are traced; every material conclusion has evidence and confidence; previous P0/P1 findings have current disposition; value, maturity, danger, and severity are separate; material query/invalidation and protected-entry patterns are assessed; strengths are preserved; weak/duplicate/dangerous/missing capabilities are separated; canonical truth and access controls precede polish; unknown external/runtime/statutory/user/provider evidence remains unverified; no certification is self-declared; and no application code, schema, dependencies, secrets, production data, or prior user-owned artifacts are modified.